

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..3

SETUP

 Copy as Markdown

Set up email sign-in

Sign in with Email verifies the user's address with a code, both when they register and when they reset their password.

CAUTION

You need to install the auth module before you continue, see [Setup](#).

Server-side configuration

Newly generated projects already configure the email identity provider in `pod.initializeAuthServices()` in your main `server.dart` file:

```
pod.initializeAuthServices(  
  tokenManagerBuilders: [  
    JwtConfigFromPasswords(),  
  ],  
  identityProviderBuilders: [  
    ServerpodCloudEmailIdpConfig(  
      appDisplayName: 'My App',  
    ),  
  ],  
);
```

Set `appDisplayName` to the name recipients should see in the verification emails. In the `development` and `test` run modes the codes are written to the server log instead of being sent, so you can complete the flow locally. In the `staging` and `production` run modes, the codes are sent as email through the Serverpod Cloud email service.

Sending is authenticated with the `scCloudAuthEmailKey` password, which Serverpod Cloud sets at deploy. The password is read when an email is sent rather than at startup, so a self-hosted server still starts



Ask AI

without it, but it cannot deliver any codes. If you host the server yourself, switch to [your own email provider](#) before you go to staging or production.

If a code cannot be sent, the failure is recorded in the session log and the sign-in flow continues unchanged, so check your [server logs](#) when a user reports a missing code.

Expose the endpoints

Newly generated projects already include the email endpoint at

`lib/src/auth/email_idp_endpoint.dart` and the migration that initializes the database, so running `serverpod start` is all that is needed.

If you are adding the auth module to an existing project, extend the abstract endpoint yourself. Create the file anywhere under your server's `lib/` directory (for example, `<project>_server/lib/src/endpoints/`). The generator picks it up:

```
import 'package:serverpod_auth_idp_server/providers/email.dart';

class EmailIdpEndpoint extends EmailIdpBaseEndpoint {}
```

Then start the server with `serverpod start` to generate the client code, and create and apply the migration that initializes the database for the provider (in the `serverpod start` terminal, press **M**). More detailed instructions can be found in the general [identity providers setup section](#).

Use your own email provider

Serverpod Cloud delivery is there to get sign-in working quickly, and it sends a standard message carrying your `appDisplayName`. You might prefer using a custom email provider to have full control over the body, layout, and language of the emails. For servers hosted outside of Serverpod Cloud, it is the only option.

Changing the email provider is done by replacing `ServerpodCloudEmailIdpConfig` with `EmailIdpConfigFromPasswords`, which requires you to pass your own callbacks for the two codes. One convenient option is the [mailer](#) package, which can send emails through any SMTP service. Most email providers, such as Resend, SendGrid, or Mandrill, support SMTP.

```
import 'package:serverpod_auth_idp_server/core.dart';
import 'package:serverpod_auth_idp_server/providers/email.dart';

import 'src/generated/serverpod.dart';

void run(List<String> args) async {
  final pod = Serverpod(args);

  pod.initializeAuthServices(
    tokenManagerBuilders: [
      JwtConfigFromPasswords(),
    ],
  );
}
```

```

identityProviderBuilders: [
    // Configure the Email Identity Provider
    // This is the basic configuration for the Email IDP to work.
    EmailIdpConfigFromPasswords(
        // Callback to send the registration verification code to the user.
        sendRegistrationVerificationCode: _sendRegistrationCode,
        // Callback to send the password reset verification code to the user.
        sendPasswordResetVerificationCode: _sendPasswordResetCode,
    ),
],
);

await pod.start();
}

void _sendRegistrationCode(
    Session session, {
    required String email,
    required UuidValue accountRequestId,
    required String verificationCode,
    required Transaction? transaction,
}) {
    // NOTE: Here you call your mail service to send the verification code to
    // the user. For testing, we will log the verification code.
    session.log('[EmailIDP] Registration code ($email): $verificationCode');
}

void _sendPasswordResetCode(
    Session session, {
    required String email,
    required UuidValue passwordResetRequestId,
    required String verificationCode,
    required Transaction? transaction,
}) {
    // NOTE: Here you call your mail service to send the verification code to
    // the user. For testing, we will log the verification code.
    session.log('[EmailIDP] Password reset code ($email): $verificationCode');
}

```

Basic configuration options

- **sendRegistrationVerificationCode**: A callback that will be called to send the registration verification code to the user. Here you should call the email sending service to send the verification code to the user.
- **sendPasswordResetVerificationCode**: A callback that will be called to send the password reset verification code to the user. Here you should call the email sending service to send the verification code to the user.

Both configurations hash passwords and verification codes with a secret pepper, which

EmailIdpConfigFromPasswords reads from the **emailSecretHashPepper** key in **config/passwords.yaml** or the **SERVERPOD_PASSWORD_emailSecretHashPepper** environment

variable. Newly generated projects already have a value for every run mode. To pass the pepper directly instead, use `EmailIdpConfig` with its `secretHashPepper` parameter.

For more details on configuration options, such as customizing password requirements, verification code generation, rate limiting, and more, see the [configuration section](#).

Client-side configuration

If you have configured the `SignInWidget` as described in the [setup section](#), the Email identity provider will be automatically detected and displayed in the sign-in widget.

You can also use the `EmailSignInWidget` to include the email authentication flow in your own custom UI.

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

EmailSignInWidget(
  client: client,
  onAuthenticated: () {
    // Do something when the user is authenticated.
    //
    // NOTE: You should not navigate to the home screen here, otherwise
    // the user will have to sign in again every time they open the app.
  },
  onError: (error) {
    // Handle errors
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error: $error')),
    );
  },
)
```

The widget automatically handles:

- Login with email and password.
- Registration with terms acceptance and email verification.
- Password reset flow with email verification.
- Navigation between screens.

For details on how to display the email authentication UI in your Flutter app, see the [customizing the UI section](#).